

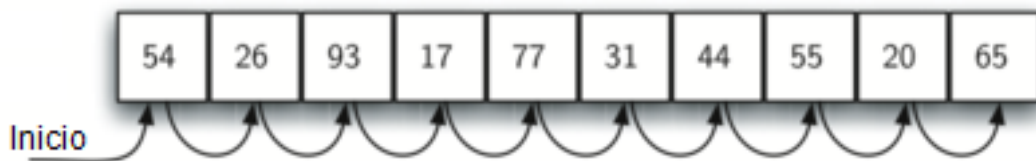
## Búsqueda y recuperación de datos

La búsqueda tiene por objetivo la localización de datos en las distintas estructuras de datos.

### Método de búsqueda secuencial

También es conocido como búsqueda lineal. Este método comienza en el primer ítem o elemento de la lista, simplemente se traslada de un ítem a otro, siguiendo el orden secuencial subyacente hasta encontrar lo que se busca o puede pasar que se queda sin ítem y no se encontró lo que se buscaba. En el caso de quedarse sin ítem, se puede decir que lo que se busca no se encuentra dentro de la lista.

En la siguiente imagen se puede comprender de mejor manera como es que funciona este método, se va recorriendo la estructura de datos iniciando por el dato que se encuentra en el inicio y se va a seguir pasando de al siguiente hasta encontrar lo que se esté buscando:



Una manera en código sería la siguiente:

```
1 def busquedaSecuencial(unaLista, item):
2     pos = 0
3     encontrado = False
4
5     while pos < len(unaLista) and not encontrado:
6         if unaLista[pos] == item:
7             encontrado = True
8         else:
9             pos = pos+1
10    return encontrado
11
12 listaPrueba = [1, 2, 32, 8, 17, 19, 42, 13, 0]
13 print(busquedaSecuencial(listaPrueba, 3))
14 print(busquedaSecuencial(listaPrueba, 13))
```

Realmente hay varias maneras de emplear este método de búsqueda, esta es solo una, lo que se hace es crear una función que va a necesitar una lista y el ítem (dato) a buscar, va a devolver un valor booleano que nos va indicar si lo que estamos buscando se encuentra en este caso en la lista. Entonces la variable “encontrado”

se va a declarar como False y si se llega a encontrar el dato buscado entonces “encontrado” será True.

## Método de búsqueda binaria

La búsqueda binaria es un algoritmo eficiente que sirve para encontrar un dato en una lista ordenada de elementos. Prácticamente funciona al dividir a la mitad la porción de la lista que podría contener al elemento buscado, este proceso se repite hasta reducir las ubicaciones posibles a una sola.

Entonces básicamente, se divide la lista o arreglo en dos mitades seguidamente se hace la comparación del elemento que estamos buscando con el elemento que se encuentra en el medio. Si el elemento del medio es igual al que estamos buscando pues se devuelve el elemento que está en el medio, de lo contrario, dependiendo del valor del dato que estemos buscando nos vamos a mover ya sea a la mitad izquierda o a la mitad derecha.

En código sería de la siguiente manera:

```
1  def busqueda_binaria(arr, x, n):
2      izquierda = 0
3      derecha = n - 1
4      medio = 0
5
6      while izquierda <= derecha:
7          medio = (derecha + izquierda) // 2
8          if arr[medio] < x:
9              izquierda = medio + 1
10         elif arr[medio] > x:
11             derecha = medio - 1
12         else:
13             return medio
14     return -1
15
16 arr = [ 2, 3, 4, 1, 5 ]
17 x = 4
18 n = len(arr)
19 resultado = busqueda_binaria(arr, x ,n)
20
21 if resultado == -1:
22     print("Elemento no encontrado")
23 else:
24     print("El elemento se encuentra en el indice: ", str(resultado))
```

### ¿Qué método de búsqueda es más eficiente?

Pienso que los dos métodos tienen sus ventajas como desventajas, entonces a mi parecer realmente depende de la situación en que se vayan a utilizar, si es una lista o arreglo que no tiene muchos datos es mejor utilizar el método secuencial, esto porque es más fácil de comprender y utilizar. Ahora si es una lista o arreglo con demasiados datos es mejor el método binario, pero hay que mencionar que si posiblemente es un poquito más complicado de comprender, pero si se tiene bien claro como funciona es el método que nos va a proporcionar más velocidad y a la computadora se puede decir que le dará menos trabajo. Ya que por ejemplo si se tienen 1000 datos con la secuencia en el peor de los casos tendría que visitar las 1000 posiciones, ahora con la binaria serían menos.

## Ordenamiento de datos

### Método de ordenamiento burbuja

También es conocido como el método del intercambio directo. Funciona revisando cada elemento de la lista que se desea ordenar con el siguiente, intercambia de posición si están en el orden equivocado, es necesario revisar varias veces la lista y repetir el procedimiento cuantas veces se necesario, es decir, que la lista quede completamente ordenado.

Para comprender de mejor manera:

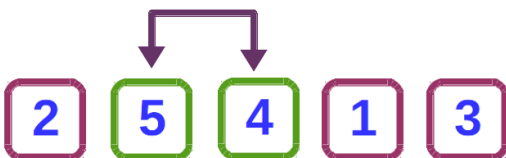
Se inicia con una lista desordenada

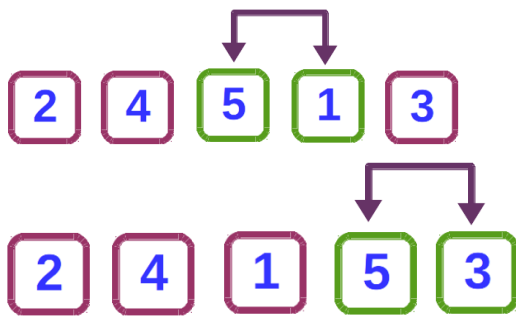


Se toman los primeros datos y se comparan, si no están ordenados entonces deben cambiar de posición

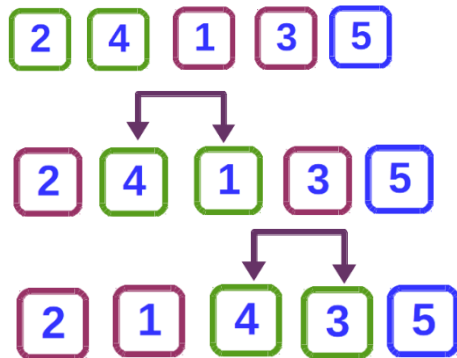


Y este proceso se repite hasta el final, es decir hasta que la lista quede completamente ordenada

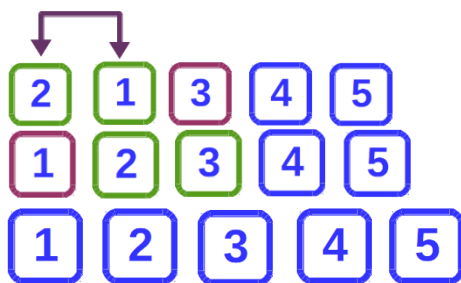




Lo que cambia al final, es que como el último número ya queda ordenado ya no se vuelve a evaluar entonces el proceso se hace más corto



Conforme los datos ya van quedando entonces ya no se vuelven a comparar



Una forma de hacer en Python sería:

```

1  def burbuja(A):
2      for i in range(1,len(A)):
3          for j in range(0,len(A)-i):
4              if(A[j+1] < A[j]):
5                  aux=A[j]
6                  A[j]=A[j+1]
7                  A[j+1]=aux
8          print (A)
9
10 A=[6,5,3,1,8,7,2,4]
11 print (A)
12 burbuja(A)
13

```

### Método de ordenamiento por inserción

Funciona básicamente en el recorrido por la lista seleccionando en cada iteración un valor como clave o de referencia y se va a ir comparando con el resto insertando en el lugar correspondiente.

Si se tiene un arreglo [5, 2, 4, 1, 3]

Entonces se toma de referencia o clave el 5 y se va a ir comparando e insertando en este caso cada número en el orden que le corresponde

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 5 | 4 | 1 | 3 |
| 2 | 4 | 5 | 1 | 3 |
| 1 | 2 | 4 | 5 | 3 |
| 1 | 2 | 3 | 4 | 5 |
| 1 | 2 | 3 | 4 | 5 |

En Python sería de la siguiente manera:

```

1  #Función para mostrar estado de la lista
2  def mostrarLista(lista, lon):
3      listaordenada=""
4      for i in range(0,lon):
5          listaordenada+=str(lista[i])+" "
6      print(listaordenada)
7
8  arreglo = [5,2,4,1,3];
9  #Recorrer el arreglo
10 for i in range(1,len(arreglo)):
11     clave = arreglo[i]
12     j = i-1
13     #Comparar el valor seleccionado con todos los valores anteriores
14     while (j>=0 and arreglo[j] > clave):
15         #Insertar el valor donde corresponda
16         arreglo[j+1] = arreglo[j]
17         j = j-1
18     arreglo[j+1] = clave
19     mostrarLista(arreglo, len(arreglo))
20 mostrarLista(arreglo, len(arreglo))

```

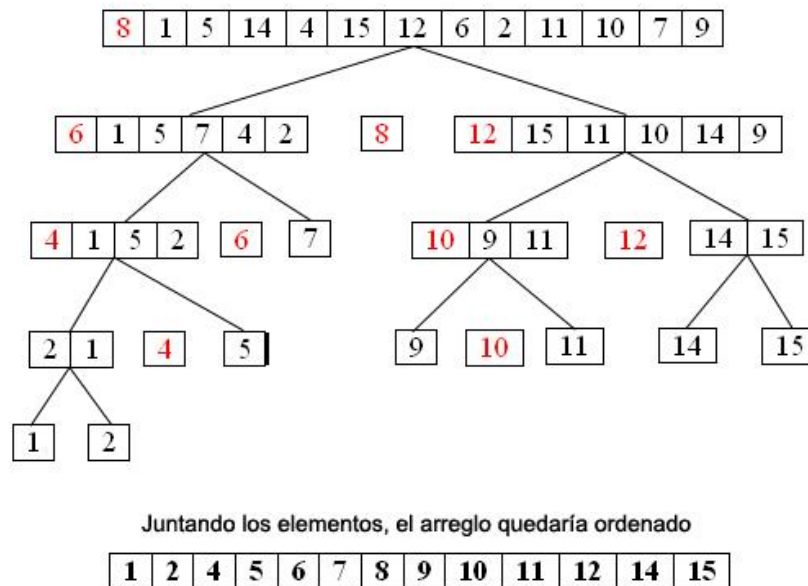
### Método de ordenamiento Quicksort

Es también conocido con el nombre de método rápido y ordenamiento por partición. La principal característica de este método es que proporciona más velocidad al programa, se basa básicamente en la técnica de divide y vencerás.

Prácticamente funciona así:

1. Se elige un elemento de una lista la cual recibe el nombre de pivote.
2. Se divide la lista en tres listas.
3. En la primera lista se almacenan los datos de la lista original que son menores o iguales al elemento de referencia, sin incluir al elemento de referencia.
4. En la segunda lista solo se almacena el dato de referencia.
5. En la tercera lista se almacenan los elementos de la lista original menos el de referencia y solo los mayores o iguales al de referencia.

6. Luego de tener estas tres listas se aplica recursión en la primera y tercera lista.
7. Ya para finalizar se deben unir las tres listas, ya que la primera y tercera estarán ordenadas solo queda unirla con la segunda que básicamente es el elemento de referencia.



Ya traducido a código sería de la siguiente manera:

```

1  milista = [9, 7, 1, 8, 4, 2, 10, 3, 6, 5]
2
3  def sort(lista):
4      izquierda = []
5      centro = []
6      derecha = []
7      if len(lista) > 1:
8          pivote = lista[0]
9          for i in lista:
10             if i < pivote:
11                 izquierda.append(i)
12             elif i == pivote:
13                 centro.append(i)
14             elif i > pivote:
15                 derecha.append(i)
16         return sort(izquierda)+centro+sort(derecha)
17     else:
18         return lista
19
20 print(milista)
21 print(sort(milista))

```

¿Qué método de ordenamiento es más eficiente?

A mi parecer el más recomendado sería el método Quicksort, si bien es cierto que hay que crear como que sublistas el proceso es más rápido, a diferencia de los otros dos métodos que básicamente los otros dos métodos se tienen que hacer demasiadas comparaciones, con el método que Quicksort igual se hacen comparaciones, pero al dividir los datos esto lo hace más fácil.

## Bibliografía

- <https://runestone.academy/runestone/static/pythoned/SortSearch/LaBusquedaSecuencial.html#st-seqsearchpython>
- <https://uneginginf05.es.tl/M-e2-todo-de-Busqueda-Secuencial-y-Binaria.htm>
- <https://es.khanacademy.org/computing/computer-science/algorithms/binary-search/a/binary-search#:~:text=La%20b%C3%BAsqueda%20binaria%20es%20un,ubicaciones%20posibles%20a%20solo%20una>
- <https://www.delftstack.com/es/howto/python/python-binary-search/>
- <https://juncotic.com/ordenamiento-de-burbuja-algoritmos-de-ordenamiento/>
- [https://es.wikipedia.org/wiki/Ordenamiento de burbuja](https://es.wikipedia.org/wiki/Ordenamiento_de_burbuja)
- <https://cdklhph.wordpress.com/2015/08/08/ordenamiento-burbuja/>
- <https://juncotic.com/ordenamiento-por-insercion-algoritmos-de-ordenamiento/>

- <https://altocodigo.blogspot.com/2018/06/ordenacion-quicksort-en-python.html>
- [http://cidecame.uaeh.edu.mx/lcc/mapa/PROYECTO/libro9/mtodo\\_quick\\_sort.html](http://cidecame.uaeh.edu.mx/lcc/mapa/PROYECTO/libro9/mtodo_quick_sort.html)